

Intégration & déploiement continu

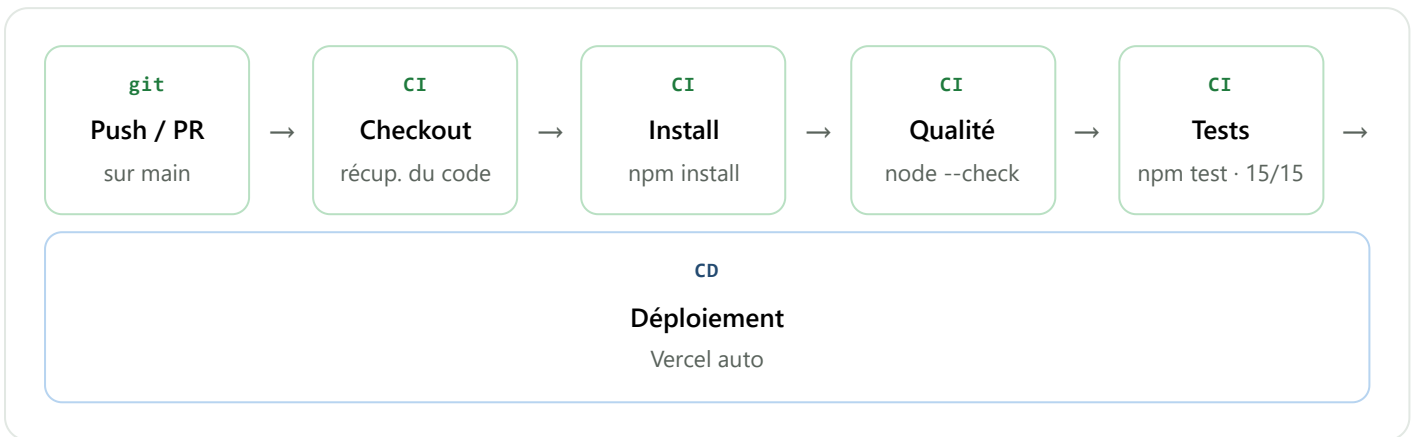
Chaîne CI/CD de **Petit Rayon de Soleil** : ce qui est automatisé aujourd'hui, ce qui reste manuel, et les évolutions prévues. Un pipeline GitHub Actions valide chaque modification avant qu'elle n'atteigne la production.

Projet Petit Rayon de Soleil **Auteur** de Breyne Hélió **Serveur CI** GitHub Actions

Déploiement Vercel (continu)

1 Le pipeline en un coup d'œil

À chaque `git push` (ou pull request) sur `main`, GitHub Actions exécute automatiquement cette chaîne. Si une étape échoue, la fusion est signalée en rouge et le déploiement ne suit pas.



Le fichier de workflow (`.github/workflows/ci.yml`) est court et versionné avec le code :

`.github/workflows/ci.yml`

```
name: CI
on:
  push:      { branches: [main] }
  pull_request: { branches: [main] }

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: npm install
      - run: node --check server.js db.js auth.js    # qualité
      - run: npm test                                # 15 tests
```

2 Automatisé, manuel, à venir

AUTOMATISÉ AUJOURD'HUI

Ce qui tourne tout seul

- **Tests** à chaque push/PR via GitHub Actions (15 tests).
- **Contrôle qualité** : vérification syntaxique de chaque fichier (`node --check`).
- **Garde-fou avant déploiement** : le hook `predeploy` relance `npm test` avant `vercel --prod` .
- **Déploiement** : Vercel redéploie à chaque push sur `main` .

ENCORE MANUEL

Ce qui reste à la main

- **Revue de code** : relecture humaine avant fusion d'une PR.
- **Migrations de base** : `init-db` / `db:reset` lancés manuellement.
- **Tests de bout en bout navigateur** : vérification visuelle de l'interface.
- **Décision de mise en production** : le choix de pousser sur `main` reste humain.

ÉVOLUTIONS PRÉVUES

Ce que je mettrais ensuite

- **ESLint + Prettier** dans le pipeline pour un vrai linting de style.
- **Couverture de code** (`node --test --experimental-test-coverage`) avec seuil minimal.
- **Matrice de versions** Node (18 / 20 / 22) pour tester la compatibilité.
- **Déploiement conditionné au vert CI** via l'intégration GitHub↔Vercel (« `deploy on success` »).

3 Interpréter un rapport d'intégration continue

Après chaque push, le rapport est consultable dans l'onglet **Actions** du dépôt GitHub. Voici comment le lire.

GitHub Actions – journal du job « test »

✓ Récupérer le code

(2s)

✓ Installer Node.js

(4s)

✓ Installer les dépendances

(9s)

✓ Contrôle qualité

(1s)

✓ Exécuter les tests

(6s)

i tests 15 · pass 15 · fail 0

Job terminé : succès → pastille verte sur le commit

Signal	Signification	Action attendue
 vert	Toutes les étapes ont réussi (installation, qualité, tests).	La fusion / le déploiement peut se faire en confiance.
 rouge	Une étape a échoué — le plus souvent un test cassé.	Ouvrir le journal, lire la ligne <code>fail</code> , corriger, repousser.
 jaune	Le pipeline est en cours d'exécution.	Attendre le résultat avant de fusionner.

La clé de lecture est la ligne finale des tests : `pass 15 · fail 0` signifie que les 15 tests passent. Un `fail` non nul pointe directement le test en échec, dont le nom et l'assertion apparaissent juste au-dessus dans le journal.

4 Pourquoi Vercel automatise déjà le déploiement

Vercel est connecté au dépôt GitHub. Cette liaison assure un **déploiement continu** sans serveur à gérer :

- À chaque `git push` sur `main`, Vercel **reconstruit et remet en ligne** l'application automatiquement — c'est le « CD » de la chaîne.
- Chaque pull request reçoit une **URL de prévisualisation** dédiée, pour tester la version avant fusion.
- Un déploiement raté peut être **annulé (rollback)** en un clic vers la version précédente.

Complémentarité CI + CD. GitHub Actions joue le rôle de *gardien de la qualité* (il teste), Vercel celui de *livreur* (il déploie). Ensemble, ils forment une chaîne où une modification passe automatiquement des tests à la mise en ligne, la décision humaine se limitant à valider et à pousser le code.

Petit Rayon de Soleil — Annexe « CI/CD ». Pipeline `.github/workflows/ci.yml` ; étapes qualité et tests vérifiées localement (15/15) avant documentation.